

Cuaderno 10 - Pipelines y GridSearchCV

En este cuaderno vamos a trabajar con una herramienta fundamental para organizar proyectos de Machine Learning: los **pipelines**.

Hasta ahora, en los cuadernos anteriores, realizamos muchas etapas por separado: cargar datos, limpiar columnas, separar variables predictoras y variable objetivo, escalar variables numéricas, codificar variables categóricas, entrenar modelos y evaluar resultados.

Ese enfoque es útil para aprender cada paso, pero cuando los proyectos crecen puede volverse difícil de mantener. Si cada transformación se aplica manualmente, aumenta el riesgo de cometer errores, repetir código o aplicar pasos distintos entre entrenamiento y prueba.

En esta clase vamos a reorganizar ese flujo de trabajo utilizando herramientas de **scikit-learn** :

- **ColumnTransformer** , para aplicar transformaciones diferentes a columnas numéricas y categóricas.
- **Pipeline** , para unir el preprocesamiento y el modelo en un único objeto.
- **GridSearchCV** , para probar distintas combinaciones de hiperparámetros de manera automática.

El objetivo no será descubrir un dataset desde cero, sino tomar un caso ya trabajado previamente y usarlo para aprender una forma más ordenada, reproducible y profesional de construir modelos.

Objetivos del cuaderno

Al finalizar este cuaderno, deberías poder:

- Explicar qué es un pipeline y por qué resulta útil en Machine Learning.
 - Comprender qué es el **data leakage** y por qué puede producir resultados engañosos.
 - Construir un preprocesamiento con **ColumnTransformer** .
 - Integrar preprocesamiento y modelo dentro de un **Pipeline** .
 - Evaluar un pipeline completo.
 - Usar **GridSearchCV** para buscar mejores hiperparámetros.
 - Utilizar el pipeline final para predecir valores sobre casos nuevos que no forman parte del dataset original.
-

Dataset utilizado

Vamos a trabajar con el dataset de laptops usado en un cuaderno anterior. El objetivo será predecir el precio de una laptop a partir de distintas características técnicas, como marca, memoria RAM, tamaño de pantalla, tipo de almacenamiento, sistema operativo y peso.

Como el dataset ya fue explorado anteriormente, en este cuaderno recuperaremos solamente las etapas necesarias para preparar el caso de trabajo. El foco principal estará puesto en la construcción del pipeline y en la automatización del flujo de Machine Learning.

¿Qué problema resuelve un pipeline?

En un proyecto de Machine Learning no alcanza con entrenar un modelo. Antes de llegar a ese punto suelen ocurrir varias etapas:

- completar valores faltantes,
- escalar variables numéricas,
- codificar variables categóricas,
- entrenar el modelo,
- hacer predicciones,
- evaluar resultados.

Cuando hacemos esos pasos manualmente, el código puede volverse largo, repetitivo y propenso a errores. Por ejemplo, podríamos escalar el conjunto de entrenamiento y olvidarnos de aplicar el mismo escalado al conjunto de prueba. O podríamos modificar una transformación en una parte del código y olvidarnos de actualizarla en otra.

Un **pipeline** permite encadenar todas esas etapas en un único objeto. Los datos entran por el inicio del pipeline, pasan por las transformaciones necesarias y finalmente llegan al modelo.

Podemos pensarlo como una línea de producción:

- 1 . entran los datos crudos,
- 2 . se transforman de manera ordenada,
- 3 . llegan al modelo,
- 4 . salen las predicciones.

La ventaja principal es que el flujo completo queda organizado, es más fácil de reutilizar y reduce el riesgo de aplicar transformaciones de manera inconsistente.

¿Qué es el data leakage?

El **data leakage**, o fuga de información, ocurre cuando información que no debería estar disponible durante el entrenamiento termina influyendo en el modelo.

Dicho de otra manera: el modelo recibe, directa o indirectamente, información del conjunto de prueba o de datos futuros. Esto puede hacer que las métricas parezcan mejores de lo que realmente son.

Un ejemplo frecuente ocurre durante el escalado.

Supongamos que aplicamos `StandardScaler` sobre todo el dataset antes de separar los datos en entrenamiento y prueba. En ese caso, el escalador calcula la media y la desviación estándar usando todos los registros, incluidos los que después formarán parte del conjunto de prueba.

Eso parece un detalle menor, pero no lo es: el conjunto de prueba debe simular datos nuevos, datos que el modelo todavía no conoce. Si usamos información del conjunto de prueba para preparar las transformaciones, la evaluación deja de ser completamente honesta.

La forma correcta es:

- 1 . separar primero los datos en entrenamiento y prueba;
- 2 . ajustar las transformaciones solamente con los datos de entrenamiento;
- 3 . aplicar esas transformaciones al conjunto de prueba usando los valores aprendidos en entrenamiento.

Los pipelines ayudan a respetar este orden automáticamente. Cuando entrenamos un pipeline, las transformaciones se ajustan sobre el conjunto de entrenamiento. Cuando hacemos predicciones, el pipeline aplica esas mismas transformaciones a los datos nuevos sin volver a ajustarlas.

Recuperación del dataset de laptops

Para este cuaderno vamos a retomar el dataset de laptops trabajado anteriormente.

Cada fila representa una laptop y sus características técnicas. La variable objetivo será `Price`, es decir, el precio del equipo.

En el cuaderno anterior exploramos el dataset con más detalle. En esta oportunidad vamos a recuperar solo los pasos necesarios para llegar rápidamente al punto central de esta clase: construir un flujo completo de preprocesamiento y modelado usando pipelines.

```
# -----
# Preparación del entorno de trabajo
# -----

# Librerías para manipulación de datos
import pandas as pd
import numpy as np

# Librerías para visualización
import matplotlib.pyplot as plt

# Librerías para descargar y ubicar archivos
import kagglehub
import os
```

```
# -----
# Descarga y carga del dataset
# -----

# Descargamos el dataset desde Kaggle
ruta_dataset = kagglehub.dataset_download("nabihazahid/laptop-details-
dataset")

# Definimos la ruta del archivo CSV
ruta_csv = os.path.join(ruta_dataset, "laptop_scrap_data.csv")

# Cargamos el archivo en un DataFrame
df = pd.read_csv(ruta_csv)

# Mostramos las primeras filas
df.head(2)
```

Using Colab cache for faster access to the 'laptop-details-dataset' dataset.

	Company	TypeName	Inches	ScreenResolution	Cpu	Gpu	OpSys	TouchS
0	MSI	MSI Prestige 1 6 AI+	1 6 . 0	2 8 8 0 x 1 8 0 0	Intel Core Ultra X 7 3 5 8 H	Intel Arc B 3 9 0	Windows	
1	MSI	MSI Prestige 1 6 AI+	1 6 . 0	2 8 8 0 x 1 8 0 0	Intel Core Ultra 9 3 8 6 H	Intel Graphics (4 - Cores)	Windows	

2 rows x 2 1 columns

Verificación rápida del dataset

Antes de empezar a preparar los datos, vamos a revisar la estructura general del DataFrame.

En esta celda veremos:

- cuántas filas y columnas tiene el dataset,
- cuáles son los nombres de las columnas,
- qué tipos de datos reconoce Pandas en cada una.

Esta revisión será breve, porque el dataset ya fue trabajado anteriormente. La hacemos solamente para ubicarnos y preparar las próximas transformaciones.

```
# -----  
# Verificación rápida del dataset  
# -----  
  
# Mostramos la cantidad de filas y columnas  
print("Dimensiones del dataset:")  
print(df.shape)  
  
print("\nColumnas del dataset:")  
print(df.columns)  
  
print("\nInformación general:")  
df.info()
```

Dimensiones del dataset:
(1563, 21)

Columnas del dataset:
Index(['Company', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu', 'Gpu',
 'OpSys', 'TouchScreen', 'Ips', 'X_res', 'Y_res', 'ppi',
 'Dedicated_Gpu',
 'Ram_GB', 'Weight_kg', 'SSD', 'HHD', 'Storage_Type',
 'Total_Storage_GB',
 'Storage_Category', 'Price'],
 dtype='object')

Información general:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1563 entries, 0 to 1562
Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	Company	1560 non-null	object
1	TypeName	1560 non-null	object
2	Inches	1560 non-null	float64
3	ScreenResolution	1560 non-null	object
4	Cpu	1560 non-null	object
5	Gpu	1560 non-null	object
6	OpSys	1560 non-null	object
7	TouchScreen	1560 non-null	float64
8	Ips	1560 non-null	float64
9	X_res	1560 non-null	float64
10	Y_res	1560 non-null	float64
11	ppi	1560 non-null	float64
12	Dedicated_Gpu	1560 non-null	float64
13	Ram_GB	1560 non-null	float64
14	Weight_kg	1560 non-null	float64
15	SSD	1560 non-null	float64
16	HHD	1560 non-null	float64
17	Storage_Type	1560 non-null	object
18	Total_Storage_GB	1560 non-null	float64
19	Storage_Category	1560 non-null	object
20	Price	1560 non-null	float64

dtypes: float64(13), object(8)
memory usage: 256.6+ KB

Limpieza mínima del dataset

El dataset tiene algunas filas con valores faltantes. Como son muy pocas en relación con el tamaño total del dataset, vamos a eliminarlas para simplificar el trabajo.

En este cuaderno no vamos a profundizar en estrategias de tratamiento de valores faltantes, porque el foco estará puesto en la construcción del pipeline.

En esta celda vamos a:

- crear una copia del DataFrame original;
- eliminar filas con valores faltantes;
- verificar las nuevas dimensiones del dataset.

```
# -----  
# Limpieza mínima del dataset  
# -----  
  
# Creamos una copia para no modificar directamente el DataFrame original  
df_limpio = df.copy()  
  
# Eliminamos filas con valores faltantes  
df_limpio = df_limpio.dropna()  
  
# Verificamos las dimensiones luego de la limpieza  
print("Dimensiones originales:", df.shape)  
print("Dimensiones luego de eliminar valores faltantes:", df_limpio.shape)  
  
# Verificamos que ya no haya valores nulos  
print("\nCantidad total de valores nulos:")  
print(df_limpio.isnull().sum().sum())
```

Dimensiones originales: (1563, 21)

Dimensiones luego de eliminar valores faltantes: (1560, 21)

Cantidad total de valores nulos:

0

Selección de variables para el modelo

Ahora vamos a seleccionar las columnas que usaremos para construir el modelo.

La variable objetivo será:

- **Price** : precio de la laptop.

Como variables predictoras vamos a usar una combinación de columnas numéricas y categóricas ya preparadas.

Entre las variables numéricas incluiremos características como tamaño de pantalla, resolución, memoria RAM, peso y almacenamiento. Entre las variables categóricas incluiremos marca, sistema operativo y tipo de almacenamiento.

Esta selección nos permitirá mostrar claramente el uso de `ColumnTransformer`, porque necesitaremos aplicar tratamientos diferentes según el tipo de columna.

```
# -----  
# Selección de variables predictoras y objetivo  
# -----  
  
# Columnas numéricas seleccionadas  
columnas_numericas = [  
    "Inches",  
    "TouchScreen",  
    "Ips",  
    "X_res",  
    "Y_res",  
    "ppi",  
    "Dedicated_Gpu",  
    "Ram_GB",  
    "Weight_kg",  
    "SSD",  
    "HHD",  
    "Total_Storage_GB"  
]  
  
# Columnas categóricas seleccionadas  
columnas_categoricas = [  
    "Company",  
    "OpSys",  
    "Storage_Type",  
    "Storage_Category"  
]  
  
# Variable objetivo  
variable_objetivo = "Price"  
  
# Armamos X e y  
X = df_limpio[columnas_numericas + columnas_categoricas]  
y = df_limpio[variable_objetivo]  
  
# Verificamos dimensiones  
print("Dimensiones de X:", X.shape)  
print("Dimensiones de y:", y.shape)  
  
print("\nColumnas numéricas seleccionadas:")  
print(columnas_numericas)  
  
print("\nColumnas categóricas seleccionadas:")  
print(columnas_categoricas)  
  
print("\nPrimeras filas de X:")  
display(X.head())  
  
print("\nPrimeros valores de y:")  
display(y.head())
```


Dimensiones de X: (1560, 16)
Dimensiones de y: (1560,)

Columnas numéricas seleccionadas:
['Inches', 'TouchScreen', 'Ips', 'X_res', 'Y_res', 'ppi', 'Dedicated_Gpu', 'Ram_GB', 'Weight_kg', 'SSD', 'HHD', 'Total_Storage_GB']

Columnas categóricas seleccionadas:
['Company', 'OpSys', 'Storage_Type', 'Storage_Category']

Primeras filas de X:

	Inches	TouchScreen	Ips	X_res	Y_res	ppi	Dedicated_Gpu
0	16.0	0.0	0.0	2880.0	1800.0	212.26	1.0
1	16.0	0.0	0.0	2880.0	1800.0	212.26	0.0
2	16.0	0.0	0.0	1920.0	1200.0	141.51	1.0
3	13.3	1.0	1.0	1920.0	1200.0	170.24	0.0
4	15.3	0.0	0.0	2560.0	1600.0	197.31	1.0

Primeros valores de y:

	Price
0	2728.8
1	2528.8
2	2428.8
3	1390.2
4	2067.8

dtype: float64

División entre entrenamiento y prueba

Ahora vamos a dividir los datos en dos conjuntos:

- `X_train` e `y_train`: datos que se usarán para entrenar el modelo.
- `X_test` e `y_test`: datos que se reservarán para evaluar el rendimiento final.

Esta división debe hacerse antes de ajustar cualquier transformación sobre los datos. De esa forma evitamos que información del conjunto de prueba influya en el entrenamiento.

Usaremos el 80 % de los datos para entrenamiento y el 20 % para prueba.

```
# -----  
# División entre entrenamiento y prueba  
# -----  
  
from sklearn.model_selection import train_test_split  
  
# Dividimos los datos en entrenamiento y prueba  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
  
# Verificamos las dimensiones obtenidas  
print("Dimensiones de X_train:", X_train.shape)  
print("Dimensiones de X_test:", X_test.shape)  
print("Dimensiones de y_train:", y_train.shape)  
print("Dimensiones de y_test:", y_test.shape)
```

```
Dimensiones de X_train: (1248, 16)  
Dimensiones de X_test: (312, 16)  
Dimensiones de y_train: (1248,)  
Dimensiones de y_test: (312,)
```

Construcción del preprocesamiento con ColumnTransformer

Nuestro dataset tiene dos tipos principales de variables:

- **Variables numéricas**, como `Ram_GB`, `Weight_kg`, `ppi` o `Total_Storage_GB`.
- **Variables categóricas**, como `Company`, `OpSys`, `Storage_Type` o `Storage_Category`.

No podemos aplicarles exactamente el mismo tratamiento.

A las variables numéricas les aplicaremos `StandardScaler`, para dejarlas en una escala comparable. En este caso usaremos un modelo basado en árboles, que no necesita obligatoriamente escalado para funcionar bien. Sin embargo, incluiremos el escalado para

construir un flujo general y reutilizable, similar al que podríamos usar con modelos que sí son sensibles a la escala, como KNN, regresión logística o modelos basados en distancias.

A las variables categóricas les aplicaremos `OneHotEncoder`, que convierte cada categoría en columnas numéricas. Por ejemplo, una columna como `Company`, que contiene valores como `Dell`, `HP`, `Lenovo` o `Apple`, no puede ser usada directamente por muchos modelos. Con `OneHotEncoder`, esas categorías se transforman en nuevas columnas con valores `0` y `1`.

Para organizar estas transformaciones usaremos `ColumnTransformer`, una herramienta de `scikit-learn` que permite aplicar distintas transformaciones a distintos grupos de columnas.

Cada transformación dentro de `ColumnTransformer` se define con una tupla de tres elementos:

```
(nombre_del_paso, transformador, columnas)
```

En nuestro caso usaremos dos pasos:

```
("numericas", StandardScaler(), columnas_numericas)
("categoricas", OneHotEncoder(handle_unknown="ignore"), columnas_categoricas)
```

Esto significa:

- al grupo de columnas numéricas se le aplicará `StandardScaler`;
- al grupo de columnas categóricas se le aplicará `OneHotEncoder`.

Además, configuraremos `OneHotEncoder` con:

```
handle_unknown="ignore"
```

Esto significa que, si más adelante aparece una categoría nueva que no estaba en los datos de entrenamiento, el pipeline no se detendrá con un error.

Por ejemplo, si el modelo fue entrenado con marcas como `Dell`, `HP` y `Lenovo`, pero luego queremos predecir el precio de una laptop de una marca que no apareció durante el entrenamiento, `handle_unknown="ignore"` permite que el pipeline siga funcionando.

Esto será importante al final del cuaderno, cuando carguemos casos nuevos manualmente para que el modelo haga predicciones.

```
# -----
# Construcción del preprocesamiento
# -----
```

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Transformador general:
# - escala columnas numéricas
# - codifica columnas categóricas
preprocesamiento = ColumnTransformer(
    transformers=[
        ("numericas", StandardScaler(), columnas_numericas),
        ("categoricas", OneHotEncoder(handle_unknown="ignore"),
columnas_categoricas)
    ]
)

# Mostramos el objeto creado
# preprocesamiento

```

Creación del pipeline completo

Ahora vamos a unir el preprocesamiento y el modelo en un solo objeto.

Hasta este momento tenemos:

- un bloque de preprocesamiento, llamado `preprocesamiento`, que sabe qué hacer con las columnas numéricas y categóricas;
- los datos separados en entrenamiento y prueba.

Lo que falta es agregar el modelo al final del flujo.

Para eso usaremos `Pipeline`, que permite encadenar pasos en orden. En este caso, el flujo será:

- 1 . aplicar el preprocesamiento;
- 2 . entrenar el modelo de regresión.

Usaremos como modelo inicial un `RandomForestRegressor`.

La ventaja de esta estructura es que, cuando entrenemos el pipeline, no tendremos que aplicar manualmente el escalado ni la codificación. El pipeline se encargará de ejecutar cada paso en el orden correcto.

```

# -----
# Creación del pipeline completo
# -----

```

```

from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor

# Creamos el modelo que usaremos al final del pipeline
modelo_rf = RandomForestRegressor(
    random_state=42,
    n_estimators=100
)

# Creamos el pipeline completo
pipeline_rf = Pipeline(
    steps=[
        ("preprocesamiento", preprocesamiento),
        ("modelo", modelo_rf)
    ]
)

# Mostramos el pipeline creado
# pipeline_rf

```

Entrenamiento y evaluación inicial del pipeline

Ahora vamos a entrenar el pipeline completo usando los datos de entrenamiento.

Cuando ejecutemos `.fit(X_train, y_train)`, el pipeline hará internamente varias cosas:

- 1 . ajustará el preprocesamiento sobre `X_train` ;
- 2 . transformará las columnas numéricas y categóricas;
- 3 . entrenará el modelo `RandomForestRegressor` con los datos ya transformados.

Luego usaremos `.predict(X_test)` para generar predicciones sobre el conjunto de prueba. En ese momento, el pipeline aplicará las mismas transformaciones aprendidas durante el entrenamiento y luego realizará la predicción.

Para evaluar el modelo usaremos tres métricas de regresión:

- `MAE` : error absoluto medio.
- `RMSE` : raíz del error cuadrático medio.
- `R2` : proporción de variabilidad explicada por el modelo.

```

# -----
# Entrenamiento y evaluación inicial del pipeline
# -----

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

```

```

# Entrenamos el pipeline completo
pipeline_rf.fit(X_train, y_train)

# Generamos predicciones sobre el conjunto de prueba
y_pred_rf = pipeline_rf.predict(X_test)

# Calculamos métricas de evaluación
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)

# Mostramos los resultados
print("Resultados del pipeline inicial con RandomForestRegressor")
print("MAE:", mae_rf)
print("RMSE:", rmse_rf)
print("R²:", r2_rf)

```

```

Resultados del pipeline inicial con RandomForestRegressor
MAE: 39.09925701923142
RMSE: 120.73442402179873
R²: 0.9642236194255476

```

Interpretación de la evaluación inicial

El pipeline inicial obtuvo un rendimiento alto sobre el conjunto de prueba.

El valor de R^2 es aproximadamente 0.96 , lo que indica que el modelo logra explicar gran parte de la variabilidad de los precios de las laptops.

También observamos los errores:

- **MAE** : indica el error promedio absoluto de las predicciones.
- **RMSE** : penaliza más los errores grandes, porque eleva los errores al cuadrado antes de calcular la raíz.

En este caso, el resultado es bueno, pero todavía corresponde tomarlo como una primera evaluación.

Hasta ahora usamos una única división entre entrenamiento y prueba. Esa división puede influir en las métricas: si el conjunto de prueba queda más fácil o más difícil, el resultado puede variar.

Por eso, el próximo paso será usar **validación cruzada**, que permite evaluar el pipeline en varias particiones diferentes del conjunto de entrenamiento.

Evaluación con validación cruzada

Ahora vamos a evaluar el pipeline usando **validación cruzada**.

La validación cruzada divide el conjunto de entrenamiento en varias partes. En cada repetición, entrena el modelo con una parte de los datos y lo valida con otra. Luego calcula un promedio de los resultados obtenidos.

Esto nos da una estimación más estable del rendimiento del modelo que una única partición train/test.

Lo importante es que vamos a aplicar la validación cruzada sobre el **pipeline completo**. Esto significa que, en cada partición, el preprocesamiento se ajustará solamente con los datos de entrenamiento de esa partición, evitando fuga de información.

```

# -----
# Evaluación del pipeline con validación cruzada
# -----

from sklearn.model_selection import cross_val_score

# Evaluamos el pipeline completo con validación cruzada
# Usamos R² como métrica de evaluación
scores_cv = cross_val_score(
    pipeline_rf,
    X_train,
    y_train,
    cv=5,
    scoring="r2"
)

# Mostramos los resultados de cada partición
print("Resultados de R² en cada partición:")
print(scores_cv)

print("\nR² promedio:")
print(scores_cv.mean())

print("\nDesvío estándar:")
print(scores_cv.std())

```

Resultados de R² en cada partición:
[0.96176462 0.95843566 0.95805072 0.9748169 0.97954774]

R² promedio:
0.9665231274553984

Desvío estándar:
0.008924792266912925

Interpretación de la validación cruzada

La validación cruzada muestra que el pipeline mantiene un rendimiento alto en distintas particiones del conjunto de entrenamiento.

Los valores de R^2 obtenidos fueron todos cercanos entre sí y el promedio fue aproximadamente 0.967.

Además, el desvío estándar fue bajo, lo que indica que el rendimiento del modelo es bastante estable. Si el desvío estándar hubiera sido muy alto, eso podría sugerir que el modelo depende demasiado de cómo se dividen los datos.

Esta evaluación es más robusta que mirar una sola división entre entrenamiento y prueba, porque prueba el pipeline varias veces sobre diferentes subconjuntos.

El punto clave es que la validación cruzada se aplicó sobre el pipeline completo. Eso significa que, en cada partición, el preprocesamiento se ajustó solamente con los datos correspondientes al entrenamiento de esa partición, evitando `data leakage`.

Introducción a GridSearchCV

Hasta ahora entrenamos un pipeline con un modelo `RandomForestRegressor` usando algunos valores definidos manualmente, por ejemplo `n_estimators=100`.

Sin embargo, muchos modelos tienen **hiperparámetros** que pueden modificar su comportamiento. En un random forest, algunos ejemplos son:

- `n_estimators` : cantidad de árboles del bosque.
- `max_depth` : profundidad máxima de cada árbol.
- `min_samples_split` : cantidad mínima de muestras necesarias para dividir un nodo.

Una forma poco práctica de buscar buenos valores sería probar combinaciones a mano. Pero eso sería lento, repetitivo y fácil de desordenar.

Para automatizar esa búsqueda usaremos `GridSearchCV`.

`GridSearchCV` prueba todas las combinaciones de hiperparámetros que le indiquemos y evalúa cada una usando validación cruzada. Al finalizar, nos informa cuál fue la mejor combinación encontrada.

Además, como nuestro modelo está dentro de un pipeline, podremos optimizar sus hiperparámetros sin separar el preprocesamiento del modelo.

Para indicar qué hiperparámetro queremos modificar, debemos usar el nombre del paso dentro del pipeline, seguido de dos guiones bajos `__`, y luego el nombre del hiperparámetro.

En nuestro pipeline, el paso final se llama `"modelo"`:

```
pipeline_rf = Pipeline(  
    steps=[  
        ("preprocesamiento", preprocesamiento),  
        ("modelo", modelo_rf)  
    ]  
)
```

Por eso, en la grilla escribiremos nombres como:

```
"modelo__n_estimators"  
"modelo__max_depth"  
"modelo__min_samples_split"
```

Esto significa:

- `modelo__n_estimators` : modificar `n_estimators` del paso llamado "modelo";
- `modelo__max_depth` : modificar `max_depth` del paso llamado "modelo";
- `modelo__min_samples_split` : modificar `min_samples_split` del paso llamado "modelo".

Esta sintaxis permite que `GridSearchCV` trabaje directamente sobre el pipeline completo.

```

# -----
# Optimización del pipeline con GridSearchCV
# -----

from sklearn.model_selection import GridSearchCV

# Definimos la grilla de hiperparámetros a probar
param_grid = {
    "modelo__n_estimators": [100, 200],
    "modelo__max_depth": [None, 10, 20],
    "modelo__min_samples_split": [2, 5]
}

# Creamos el objeto GridSearchCV
grid_search = GridSearchCV(
    estimator=pipeline_rf,
    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1
)

# Entrenamos la búsqueda sobre el conjunto de entrenamiento
grid_search.fit(X_train, y_train)

# Mostramos los mejores resultados
print("Mejores hiperparámetros encontrados:")
print(grid_search.best_params_)

print("\nMejor R² promedio en validación cruzada:")
print(grid_search.best_score_)

```

```

Mejores hiperparámetros encontrados:
{'modelo__max_depth': 10, 'modelo__min_samples_split': 2,
 'modelo__n_estimators': 200}

```

```

Mejor R² promedio en validación cruzada:
0.9667468915429289

```

Revisión de los resultados de GridSearchCV

`GridSearchCV` no solo nos informa cuál fue la mejor combinación de hiperparámetros. También guarda información sobre todas las combinaciones evaluadas.

En esta celda vamos a convertir esos resultados en un `DataFrame` para analizarlos con más comodidad.

Nos interesará ver:

- qué combinación de hiperparámetros se probó;

- qué puntaje promedio obtuvo cada combinación;
- qué tan estables fueron los resultados en validación cruzada.

Esto permite entender mejor el proceso de búsqueda, en lugar de mirar solamente el resultado final.

```
# -----  
# Revisión de resultados de GridSearchCV  
# -----  
  
# Convertimos los resultados completos de GridSearchCV en un DataFrame  
resultados_grid = pd.DataFrame(grid_search.cv_results_)  
  
# Seleccionamos algunas columnas relevantes para analizar  
columnas_resultados = [  
    "param_modelo__n_estimators",  
    "param_modelo__max_depth",  
    "param_modelo__min_samples_split",  
    "mean_test_score",  
    "std_test_score",  
    "rank_test_score"  
]  
  
# Ordenamos por ranking: 1 indica la mejor combinación  
resultados_grid_resumen = resultados_grid[columnas_resultados].sort_values(  
    by="rank_test_score"  
)  
  
# Mostramos los resultados  
display(resultados_grid_resumen)
```

	param_modelo__n_estimators	param_modelo__max_depth	param_modelo__min_samples
5	2 0 0	1 0	
0	1 0 0	None	
8	1 0 0	2 0	
1	2 0 0	None	
9	2 0 0	2 0	
4	1 0 0	1 0	
7	2 0 0	1 0	
1 1	2 0 0	2 0	
3	2 0 0	None	
6	1 0 0	1 0	
2	1 0 0	None	
1 0	1 0 0	2 0	

Interpretación de los resultados de GridSearchCV

La tabla anterior muestra todas las combinaciones de hiperparámetros evaluadas por `GridSearchCV`.

La mejor combinación encontrada fue:

- `modelo__n_estimators = 200`
- `modelo__max_depth = 10`
- `modelo__min_samples_split = 2`

El R^2 promedio de esa combinación fue aproximadamente `0.9667`.

La mejora respecto del modelo inicial es pequeña. Esto no significa que `GridSearchCV` no haya sido útil. Significa que el modelo base ya tenía un rendimiento alto y que, dentro de la grilla que probamos, no había una combinación que cambiara radicalmente el resultado.

Lo importante es que ahora la elección de hiperparámetros no se hizo “a ojo”, sino mediante una búsqueda sistemática con validación cruzada.

Además, todo el proceso se realizó sobre el pipeline completo. Esto significa que cada combinación fue evaluada aplicando correctamente el preprocesamiento dentro de cada partición de validación cruzada.

Evaluación final del mejor pipeline

`GridSearchCV` guarda automáticamente el mejor modelo encontrado en el atributo `best_estimator_`.

Ese objeto no contiene solamente el modelo final, sino el pipeline completo:

- 1 . preprocesamiento de columnas numéricas y categóricas;
- 2 . modelo `RandomForestRegressor` con los mejores hiperparámetros encontrados.

Ahora vamos a usar ese mejor pipeline para hacer predicciones sobre `X_test` y calcular las métricas finales.

Esta evaluación sobre el conjunto de prueba nos permite estimar cómo podría comportarse el modelo frente a datos nuevos.

```
# -----  
# Evaluación final del mejor pipeline  
# -----  
  
# Recuperamos el mejor pipeline encontrado por GridSearchCV  
mejor_pipeline = grid_search.best_estimator_  
  
# Generamos predicciones sobre el conjunto de prueba  
y_pred_mejor = mejor_pipeline.predict(X_test)  
  
# Calculamos métricas finales  
mae_mejor = mean_absolute_error(y_test, y_pred_mejor)  
rmse_mejor = np.sqrt(mean_squared_error(y_test, y_pred_mejor))  
r2_mejor = r2_score(y_test, y_pred_mejor)  
  
# Mostramos los resultados  
print("Resultados finales del mejor pipeline")  
print("MAE:", mae_mejor)  
print("RMSE:", rmse_mejor)  
print("R²:", r2_mejor)
```

```
Resultados finales del mejor pipeline  
MAE: 40.58299232717762  
RMSE: 119.00035701586289  
R²: 0.9652439269610764
```

Comparación entre el pipeline inicial y el pipeline optimizado

Ahora vamos a comparar el pipeline inicial con el pipeline optimizado mediante `GridSearchCV`.

Esta comparación nos permitirá ver si la búsqueda de hiperparámetros produjo una mejora clara sobre el conjunto de prueba.

Es importante recordar que no siempre una optimización mejora todas las métricas. A veces una configuración puede mejorar el `R2`, pero empeorar levemente el `MAE`, o viceversa.

Por eso conviene mirar varias métricas antes de sacar conclusiones.

```
# -----  
# Comparación entre pipeline inicial y optimizado  
# -----  
  
comparacion_pipelines = pd.DataFrame({  
    "modelo": ["Pipeline inicial", "Pipeline optimizado"],  
    "MAE": [mae_rf, mae_mejor],  
    "RMSE": [rmse_rf, rmse_mejor],  
    "R2": [r2_rf, r2_mejor]  
})  
  
display(comparacion_pipelines)
```

	modelo	MAE	RMSE	R 2
0	Pipeline inicial	3 9 . 0 9 9 2 5 7	1 2 0 . 7 3 4 4 2 4	0 . 9 6 4 2 2 4
1	Pipeline optimizado	4 0 . 5 8 2 9 9 2	1 1 9 . 0 0 0 3 5 7	0 . 9 6 5 2 4 4

Interpretación de la comparación

Los resultados del pipeline inicial y del pipeline optimizado son muy parecidos.

El pipeline optimizado obtuvo un `R2` apenas superior y un `RMSE` algo menor, lo que indica una leve mejora en la penalización de errores grandes.

Sin embargo, el `MAE` aumentó un poco, lo que significa que el error absoluto promedio fue ligeramente mayor.

Esto muestra una idea importante: `GridSearchCV` no garantiza una mejora notable en todos los casos. Su valor principal está en que permite probar combinaciones de hiperparámetros de forma ordenada, sistemática y reproducible.

En este ejemplo, el modelo inicial ya tenía un rendimiento alto. Por eso, la optimización encontró una configuración apenas mejor según la métrica usada durante la búsqueda (R^2), pero no produjo un cambio radical.

A partir de ahora, usaremos el pipeline optimizado como modelo final.

Predicción sobre casos nuevos

Hasta ahora entrenamos y evaluamos el pipeline usando datos del dataset original.

Ahora vamos a simular una situación más cercana al uso real de un modelo: cargar manualmente algunos casos nuevos y pedirle al pipeline que prediga el precio estimado.

Estos casos no forman parte del dataset original. Son laptops inventadas para probar el funcionamiento del modelo final.

La ventaja de usar un pipeline es que no necesitamos aplicar manualmente el escalado ni la codificación de variables categóricas. Solo debemos construir un DataFrame con las mismas columnas que usamos durante el entrenamiento.

El pipeline se encargará de:

- 1 . tomar las columnas numéricas;
- 2 . escalarlas según lo aprendido durante el entrenamiento;
- 3 . tomar las columnas categóricas;
- 4 . codificarlas con `OneHotEncoder` ;
- 5 . enviar los datos transformados al modelo;
- 6 . devolver la predicción final.

```
# -----  
# Creación de casos nuevos  
# -----  
  
# Creamos algunos casos nuevos inventados  
casos_nuevos = pd.DataFrame([  
    {  
        "Inches": 15.6,  
        "TouchScreen": 0,  
        "Ips": 1,
```



```

        "X_res": 1920,
        "Y_res": 1080,
        "ppi": 141.21,
        "Dedicated_Gpu": 0,
        "Ram_GB": 8,
        "Weight_kg": 1.8,
        "SSD": 256,
        "HHD": 0,
        "Total_Storage_GB": 256,
        "Company": "Lenovo",
        "OpSys": "Windows 10",
        "Storage_Type": "SSD",
        "Storage_Category": "SSD"
    },
    {
        "Inches": 14.0,
        "TouchScreen": 0,
        "Ips": 1,
        "X_res": 2560,
        "Y_res": 1600,
        "ppi": 215.63,
        "Dedicated_Gpu": 0,
        "Ram_GB": 16,
        "Weight_kg": 1.3,
        "SSD": 512,
        "HHD": 0,
        "Total_Storage_GB": 512,
        "Company": "Apple",
        "OpSys": "macOS",
        "Storage_Type": "SSD",
        "Storage_Category": "SSD"
    },
    {
        "Inches": 17.3,
        "TouchScreen": 0,
        "Ips": 0,
        "X_res": 1920,
        "Y_res": 1080,
        "ppi": 127.34,
        "Dedicated_Gpu": 1,
        "Ram_GB": 16,
        "Weight_kg": 2.8,
        "SSD": 512,
        "HHD": 1000,
        "Total_Storage_GB": 1512,
        "Company": "MSI",
        "OpSys": "Windows 10",
        "Storage_Type": "Hybrid",
        "Storage_Category": "SSD+HDD"
    }
]
)

# Mostramos los casos nuevos
display(casos_nuevos)

```

	Inches	TouchScreen	Ips	X_res	Y_res	ppi	Dedicated_Gpu	Ram_Gb
0	15.6	0	1	1920	1080	141.2	1	0
1	14.0	0	1	2560	1600	215.6	3	0
2	17.3	0	0	1920	1080	127.3	4	1

Verificación de columnas y predicción

Antes de pedirle al modelo que prediga, vamos a comprobar que el DataFrame `casos_nuevos` tiene las mismas columnas que el modelo recibió durante el entrenamiento.

Esto es importante porque el pipeline espera una estructura concreta: las mismas variables numéricas y categóricas que usamos en `X`.

Una vez verificado eso, usaremos el mejor pipeline encontrado por `GridSearchCV` para predecir el precio estimado de cada laptop nueva.

```
# -----
# Predicción sobre casos nuevos
# -----

# Verificamos que los casos nuevos tengan las mismas columnas que X
print("Columnas esperadas por el modelo:")
print(list(X.columns))

print("\nColumnas de los casos nuevos:")
print(list(casos_nuevos.columns))

# Realizamos predicciones con el mejor pipeline
predicciones_precios = mejor_pipeline.predict(casos_nuevos)

# Agregamos las predicciones al DataFrame de casos nuevos
casos_nuevos_con_prediccion = casos_nuevos.copy()
casos_nuevos_con_prediccion["Precio_estimado"] = predicciones_precios

# Mostramos el resultado
display(casos_nuevos_con_prediccion)
```

Columnas esperadas por el modelo:

```
['Inches', 'TouchScreen', 'Ips', 'X_res', 'Y_res', 'ppi', 'Dedicated_Gpu',
'Ram_GB', 'Weight_kg', 'SSD', 'HHD', 'Total_Storage_GB', 'Company', 'OpSys',
'Storage_Type', 'Storage_Category']
```

Columnas de los casos nuevos:

```
['Inches', 'TouchScreen', 'Ips', 'X_res', 'Y_res', 'ppi', 'Dedicated_Gpu',
'Ram_GB', 'Weight_kg', 'SSD', 'HHD', 'Total_Storage_GB', 'Company', 'OpSys',
'Storage_Type', 'Storage_Category']
```

	Inches	TouchScreen	Ips	X_res	Y_res	ppi	Dedicated_Gpu	Ram_GB
0	1 5.6	0	1	1 9 2 0	1 0 8 0	1 4 1.2 1	0	
1	1 4.0	0	1	2 5 6 0	1 6 0 0	2 1 5.6 3	0	1
2	1 7.3	0	0	1 9 2 0	1 0 8 0	1 2 7.3 4	1	1

Interpretación de las predicciones sobre casos nuevos

La tabla anterior muestra los casos nuevos junto con la columna `Precio_estimado`.

Cada fila representa una laptop inventada que no formaba parte del dataset original. El modelo utilizó sus características técnicas para estimar un precio.

Lo más importante de esta sección no es solamente el valor numérico de cada predicción, sino el modo en que la predicción fue realizada.

Como usamos un pipeline, no tuvimos que aplicar manualmente el escalado ni la codificación de variables categóricas. El objeto `mejor_pipeline` ya contiene todo el flujo completo:

- preprocesamiento de columnas numéricas;
- codificación de columnas categóricas;
- modelo entrenado con los mejores hiperparámetros encontrados por `GridSearchCV`.

Esto hace que el modelo sea más fácil de reutilizar. Si mañana quisiéramos estimar el precio de otra laptop, solo deberíamos crear una nueva fila con las mismas columnas y pasarla al pipeline con `.predict()`.

De todos modos, estas predicciones deben interpretarse con cuidado. El modelo aprendió a partir de los datos disponibles en el dataset. Si le pasamos una laptop muy distinta a las que vio durante el entrenamiento, la predicción puede ser menos confiable.

Conclusiones finales

En este cuaderno trabajamos con una forma más organizada y profesional de construir modelos de Machine Learning.

Partimos de un dataset ya conocido y reorganizamos el flujo de trabajo usando herramientas de `scikit-learn` que permiten integrar varias etapas en una única estructura.

A lo largo del recorrido vimos que un pipeline permite unir:

- el preprocesamiento de los datos;
- la transformación de variables numéricas y categóricas;
- el entrenamiento del modelo;
- la generación de predicciones.

También vimos que `ColumnTransformer` es especialmente útil cuando el dataset contiene distintos tipos de columnas. En nuestro caso, aplicamos `StandardScaler` a las variables numéricas y `OneHotEncoder` a las variables categóricas.

Luego usamos `Pipeline` para unir ese preprocesamiento con un modelo `RandomForestRegressor`. Esto nos permitió entrenar, evaluar y reutilizar el flujo completo sin aplicar transformaciones manuales por separado.

Más adelante incorporamos `GridSearchCV`, que nos permitió probar distintas combinaciones de hiperparámetros de manera sistemática y con validación cruzada. Aunque la mejora obtenida fue pequeña, el procedimiento fue más ordenado, objetivo y reproducible.

Finalmente, usamos el mejor pipeline para predecir precios sobre casos nuevos. Esta última etapa mostró una de las ventajas más importantes de los pipelines: una vez entrenado el flujo completo, podemos pasarle nuevos datos con las columnas originales y el pipeline se encarga internamente de transformarlos antes de hacer la predicción.

Como idea principal, un pipeline no solo simplifica el código. También ayuda a evitar errores, reduce el riesgo de `data leakage` y permite construir modelos más fáciles de evaluar, ajustar y reutilizar.

Apéndice: buenas prácticas y errores comunes

Ajustar transformaciones antes de dividir los datos

Uno de los errores más comunes es aplicar transformaciones sobre todo el dataset antes de hacer la división entre entrenamiento y prueba.

Por ejemplo, escalar todas las variables antes de separar `X_train` y `X_test` puede producir `data leakage`, porque el escalador calcula información usando también datos que deberían quedar reservados para evaluación.

La práctica correcta es dividir primero y ajustar las transformaciones solo con los datos de entrenamiento. Los pipelines ayudan a respetar ese orden.

Olvidar que los casos nuevos deben tener las mismas columnas

Cuando usamos un pipeline entrenado, los nuevos datos deben tener la misma estructura que los datos usados para entrenar.

Eso significa que el DataFrame de casos nuevos debe incluir las mismas columnas predictoras que `X`.

Si falta una columna, sobra una columna importante o cambia el nombre de alguna variable, el pipeline puede fallar o producir resultados incorrectos.

No respetar el formato de los datos originales

Además de tener las mismas columnas, los casos nuevos deben respetar el mismo criterio de carga usado en el dataset original.

Por ejemplo:

- `TouchScreen` debe indicarse como `0` o `1`.
- `Ips` debe indicarse como `0` o `1`.
- `Dedicated_Gpu` debe indicarse como `0` o `1`.
- `Ram_GB`, `SSD`, `HHD` y `Total_Storage_GB` deben cargarse en GB.
- `Weight_kg` debe cargarse en kilogramos.
- Las columnas categóricas deben escribirse con valores compatibles con los usados durante el entrenamiento.

Si los datos nuevos se cargan con otro criterio, la predicción puede perder sentido.

No manejar categorías desconocidas

En datos reales, puede aparecer una categoría nueva que no estaba en el conjunto de entrenamiento.

Por ejemplo, una marca de laptop que no apareció en el dataset original.

Por eso usamos:

```
OneHotEncoder(handle_unknown="ignore")
```

Esta configuración permite que el pipeline no se detenga si encuentra una categoría desconocida al hacer predicciones.

Confundir parámetros con hiperparámetros

Los parámetros son valores que el modelo aprende durante el entrenamiento.

Los hiperparámetros son decisiones que definimos antes de entrenar, como la cantidad de árboles de un random forest o la profundidad máxima de cada árbol.

`GridSearchCV` sirve para probar distintas combinaciones de hiperparámetros.

Nombrar mal los hiperparámetros dentro de GridSearchCV

Cuando el modelo está dentro de un pipeline, los hiperparámetros deben escribirse usando el nombre del paso, dos guiones bajos y el nombre del hiperparámetro.

Por ejemplo, si el paso del modelo se llama `"modelo"`, usamos:

```
"modelo__n_estimators"  
"modelo__max_depth"  
"modelo__min_samples_split"
```

Esta sintaxis le indica a `GridSearchCV` que esos hiperparámetros pertenecen al paso llamado `"modelo"` dentro del pipeline.